The `:partial` parameter to `render` is the name of the template to render (but without the leading underscore). This name must be both a valid filename and a valid Ruby identifier (so `a-b` and `20042501` aren't valid names for partials). The `:object` parameter identifies an object to be passed into the partial. This object will be available within the template via a local variable with the same name as the template. In this example, the `@an_article` object will be passed to the template, and the template can access it using the local variable `article`. That's why we could write things such as `article.title` in the partial.

You can set additional local variables in the template by passing `render` a `:locals` parameter. This takes a hash where the entries represent the names and values of the local variables to set:

```
render(partial: 'article',
       object:  @an_article,
       locals:  { authorized_by: session[:user_name],
                  from_ip:       request.remote_ip })
```

Partials and Collections

Applications commonly need to display collections of formatted entries. A blog might show a series of articles, each with text, author, date, and so on. A store might display entries in a catalog, where each has an image, a description, and a price.

The `:collection` parameter to `render` works in conjunction with the `:partial` parameter. The `:partial` parameter lets us use a partial to define the format of an individual entry, and the `:collection` parameter applies this template to each member of the collection.

To display a list of article model objects using our previously defined `_article.html.erb` partial, we could write this:

```
<%= render(partial: "article", collection: @article_list) %>
```